

Individual Contribution Reflection – Allen Ji (fish11)

1. Contribution Summary

Main – Admin Question Management: I built the protected admin backend APIs for question create, edit, delete, active/inactive toggle, and JSON bulk import. I also implemented the admin dashboard, question form, edit modal, and bulk import interface.

Main – Validation: I added frontend validation with React Hook Form and Zod and backend validation for question text, four options, numeric correctAnswer, active status, and optional explanation.

Main – Testing: I added admin controller tests covering access control, question creation, validation errors, bulk import, and toggle behaviour.

Supporting – Integration: I fixed frontend JSX/import issues, Vite page filename errors, and quiz start navigation to /quiz.

Supporting – Security & Robustness: I mounted the quiz submit rate limiter, protected leaderboard access, moved admin login to /bosscoming, removed the public admin link, and added ObjectId validation for quiz submission and attempt detail lookup.

Supporting – Data Consistency: I updated seed data to match the current Question schema, including questionText, numeric correctAnswer, active, and explanation.

2. Technical Challenge

The main technical challenge was keeping the question data format consistent across the backend schema, admin APIs, frontend form, bulk import, seed data, and Review Mode requirement.

Earlier versions had mismatches around questionText, correctAnswer, and explanation. The current Question model stores correctAnswer as a numeric index from 0 to 3, not as answer text. I resolved this by aligning backend validation, frontend form submission, bulk import format, and seed data with the same schema. This made admin question management more reliable and reduced runtime errors during integration testing.

3. Subsystem Diagram

 mermaid-diagram.png

4. Git Commit Evidence

Admin backend: Implemented protected admin controller and routes for question CRUD, active/inactive toggle, and JSON bulk import. Added validation for question creation, update, import, and toggle behaviour. (2479978, 284fd25)

Admin frontend: Built the admin dashboard, question form, edit modal, and bulk import interface. Also aligned admin styling and fixed React key usage. (df3c0c, fc0f700, a480298)

Data consistency: Updated seed data to match the current Question schema, including questionText, numeric correctAnswer, active status, and explanation. (ee4e681)

Frontend integration: Fixed JSX/import issues, standardised page filenames, resolved Vite import errors, and fixed quiz start navigation to /quiz. (63e5093, 1768b9e, 01a0100)

Security and robustness: Mounted the quiz submit rate limiter, protected leaderboard access, moved admin login to /bosscoming, removed the public admin link, added ObjectId validation, and improved route consistency. (c89b303, 3a77060, c8b48f6)

These commits show my ownership of the admin subsystem, with supporting work on validation, route security, frontend stability, and integration.

5. Design Reflection & Learning

The approved variation was Review Mode after completion. My admin subsystem supports this by allowing admins to manage the optional explanation field for each question. I kept it optional so existing questions and bulk imports remain valid even without explanations, while still allowing the review screen to show richer feedback when explanation data is available.

Through this subsystem, I learned how frontend validation, backend validation, database schema design, and route protection need to stay consistent in a full-stack MERN application. I also learned that small integration details, such as correct route paths, ObjectId validation, and protected endpoints, can strongly affect the reliability of the whole system. I also learned the importance of clear communication within the team, as integration work depended on consistent data formats, routes, and responsibilities.